UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Aprendizaje por Refuerzo



Proyecto Final

Supervisor de mesa de blackjack con aprendizaje por refuerzo

Francis Aguilar 22243

Gerardo Pineda 22880

Catedrático: Alberto Suriano

1. Descripción del problema

En el blackjack, un jugador puede ganarle a la casa sin hacer trampa si cuenta las cartas que ya salieron. Cuando en el zapato quedan muchas cartas altas (dieces y ases), el jugador tiene más probabilidad de ganar, así que sube su apuesta. Cuando el mazo no le conviene, apuesta lo mínimo. Por eso los casinos tienen supervisores de mesa (pit bosses) que observan a los jugadores y expulsan a los que creen que cuentan cartas.

Este trabajo es difícil por tres razones:

- El supervisor no sabe qué tipo de jugador tiene enfrente. Solo ve lo que hace: cuánto apuesta, cómo juega, si gana o pierde y qué cartas salieron.
- La información llega poco a poco, ronda por ronda. Además, un contador hábil se esconde: cambia poco su apuesta, mete ruido y comete errores a propósito.
- Cada decisión cuesta dinero. Si expulsa tarde, el contador sigue ganándole a la casa. Si expulsa a un jugador normal, el casino pierde a un cliente.

El proyecto entrena un agente de aprendizaje por refuerzo que hace de supervisor en una mesa simulada de 5 sillas. En cada ronda, para cada silla, el agente decide si no hace nada, si vigila o si expulsa al jugador. En la mesa hay tres tipos de jugador:

Tipo	Proporción	Comportamiento	Decisión correcta
Casual	70%	Apuesta por rachas o al azar y comete errores de estrategia básica.	Dejarlo jugar, es el cliente.
Contador abierto	15%	Cuenta cartas con el sistema Hi-Lo y sube mucho la apuesta cuando el mazo le conviene.	Expulsarlo.
Contador disimulado	15%	Cuenta cartas pero se esconde: sube poco la apuesta, mete ruido y comete errores a propósito.	Expulsarlo. Es el caso más difícil.

Este problema se parece a otros problemas reales de detección, como encontrar fraudes en transacciones o bots en plataformas en línea. En todos hay que decidir cuándo actuar sin tener toda la información, y equivocarse en un sentido no cuesta lo mismo que equivocarse en el otro.

2. Análisis

2.1 Por qué usar aprendizaje por refuerzo y no solo un clasificador

Un clasificador responde la pregunta "¿este jugador es contador?". El supervisor necesita responder otra pregunta: "¿lo expulso ahora o espero una ronda más?". Hay tres diferencias importantes:

- La decisión se toma en el tiempo. Expulsar pronto ahorra dinero, pero es más fácil equivocarse. Esperar da más información, pero el contador sigue ganando. Una función de valor sirve justo para estimar si vale la pena esperar.
- Lo que importa es el dinero. El objetivo es cuidar el dinero de la casa sin perder clientes, y eso se expresa como una recompensa, no como una etiqueta.

- No hay etiquetas. En un casino real nadie confirma si un jugador contaba cartas. El agente aprende solo con la recompensa. El clasificador sí usa las etiquetas del simulador, por eso sirve como punto de comparación exigente pero no se podría usar en la vida real.

2.2 Formulación como POMDP

El supervisor no ve todo el estado del juego. No sabe el tipo de cada jugador, su conteo ni cuánto dinero tiene. Solo ve su comportamiento. Por eso el problema es un proceso de decisión de Markov parcialmente observable (POMDP). Hay dos formas de manejarlo: darle memoria a la política con una red recurrente, o resumir la historia de cada jugador en variables que ya contengan lo importante. El proyecto prueba las dos.

2.3 Riesgos identificados

- Recompensa mal balanceada. Si expulsar a un casual casi no se castiga, el agente expulsa a todos. Si se castiga demasiado, no expulsa a nadie.
- Reward hacking. Al principio los contadores se quedaban más rondas en la mesa que los casuales. Así el agente podía reconocerlos por el tiempo sentados y no por su forma de jugar. Se corrigió dando a todos la misma duración promedio de 100 rondas.
- Mucho azar en el blackjack. Los resultados de cada mano varían mucho. Por eso todas las comparaciones usan las mismas 20 semillas (de la 1000 a la 1019), así cada supervisor ve exactamente las mismas manos.
- Jugar perfecto delata. Los contadores casi no cometen errores de estrategia básica y los casuales sí (0.108 contra 0.009 o menos). Esto hace más fácil detectarlos.

3. Propuesta de solución

Se construye un entorno en Gymnasium que simula la mesa completa. Sobre ese entorno se entrena un supervisor con Proximal Policy Optimization (PPO) y se compara con supervisores que no usan aprendizaje por refuerzo. La propuesta tiene cinco partes:

- Un simulador de blackjack realista: zapato de 6 mazos con penetración del 75%, reglas S17, pago 3 a 2, seguro, dobles y divisiones, más la estrategia básica y las desviaciones Illustrious 18.
- Jugadores con comportamiento comprobado. Se simularon 2 millones de rondas para verificar que el contador gana dinero, el casual pierde y el disimulado gana, aunque menos.
- Un supervisor de RL con una red que usa los mismos pesos para las 5 sillas. Se entrenó con PPO y con RecurrentPPO (que tiene memoria LSTM) para compararlos.
- Puntos de comparación (baselines) para ver si el RL realmente ayuda: una regla fija y clasificadores supervisados entrenados con las etiquetas reales.
- Extensiones: un curriculum que aumenta la dificultad del camuflaje, un adversario evolutivo (CMA-ES) que aprende a esconderse del supervisor, reentrenamiento continuo y explicación de las decisiones.

El trabajo se dividió en dos partes, conectadas por un formato común de datos (el evento SeatEvent que la mesa genera por silla en cada ronda). Gerardo Pineda hizo la simulación: el motor de blackjack, los jugadores, la mesa, el entorno, el curriculum, el adversario evolutivo, la mesa visual y la explicabilidad. Francis Aguilar hizo el supervisor: el cálculo de variables, la red de la política, el entrenamiento, los baselines, el reentrenamiento continuo, las métricas y la evaluación.

4. Descripción de la solución

4.1 Arquitectura general

El entorno CasinoTableEnv contiene la mesa de 5 sillas. En cada ronda la mesa reparte, los jugadores apuestan y juegan según su política, y la mesa genera un evento por silla. Un extractor convierte esos eventos en lo que observa el agente. El supervisor es el único que aprende. Los jugadores no aprenden en la mesa, salvo el adversario durante el reentrenamiento.

4.2 Observación

El extractor (src/env/features.py) genera una matriz de 5 por 15, con una fila por silla calculada sobre las últimas 50 rondas del jugador, y 4 valores globales de la mesa. Las variables más importantes son:

- corr_bet_tc: correlación de Pearson entre la apuesta y el true count (TC). Es la señal clásica. En las pruebas dio +0.03 para casuales, +0.58 para disimulados y +0.71 para contadores.
- rank_corr_bet_tc: la misma correlación pero con Spearman. Aguanta mejor el ruido que mete el disimulado.
- bet_spread: apuesta máxima dividida entre la mínima. Un contador necesita variar su apuesta para ganar, mientras que el casual suele apostar parejo.
- index_play_rate: cuántas veces el jugador se sale de la estrategia básica justo cuando el conteo lo justifica. El contador lo hace a propósito, el casual por casualidad.
- basic_strategy_error_rate: errores de estrategia básica por ronda. Los casuales tienen 0.108 y los contadores 0.009 o menos.
- corr_bet_prev_result: relación entre la apuesta y el resultado anterior. Es típico del casual, que apuesta por rachas y no según el mazo.
- insurance_tc_mean, sit_out_rate y entry_tc: cuándo toma seguro, cuántas rondas no apuesta y con qué conteo se sienta. El contador toma seguro con conteo alto y entra a la mesa cuando le conviene.
- rounds_seen: cuántas rondas lleva sentado. Con pocas rondas las demás variables todavía no dicen mucho.

Se usaron variables hechas a mano en lugar de las cartas crudas porque resumen 50 rondas en 15 números fáciles de entender. Así el agente aprende en una hora y no en días, sus decisiones se pueden explicar y el estado casi no necesita memoria. Todas las variables usan solo información pública. El conteo se calcula con las cartas visibles, igual que lo haría un supervisor real.

4.3 Acciones y recompensa

En cada ronda el agente toma una decisión por silla: 0 es no hacer nada, 1 es vigilar y 2 es expulsar. Se decide por silla porque puede haber varios contadores a la vez. Cuando expulsa a alguien, la silla queda libre y llega otro jugador. La recompensa refleja el objetivo económico:

Situación	Valor	Razón
Expulsar a un contador	+10	Es el objetivo.
Expulsar a un disimulado	+15	Es más difícil de detectar, por eso vale más.
Expulsar a un casual	-25	Es un cliente perdido. Pesa más que un acierto para que el agente no expulse por si acaso.
Cada ronda con un contador sentado	Menos lo que gana el contador	Esperar cuesta dinero.
Vigilar	-0.05	Un costo pequeño para que no vigile sin razón.
Detectar temprano	+20 dividido entre las rondas vistas	Premia detectar pronto, solo si acierta.

La idea es simple: expulsar tarde cuesta dinero y expulsar mal cuesta clientes. Cada episodio dura 600 rondas, como un turno del supervisor, con jugadores que llegan y se van.

4.4 Red de la política

La política (src/supervisor/policy.py) usa una misma red pequeña para las 5 sillas, porque un contador se comporta igual en cualquier asiento. Así cada jugador sirve como ejemplo de entrenamiento para la misma red, el agente aprende cinco veces más rápido y no se aprende de memoria las posiciones. Los resultados de las 5 sillas se juntan con los datos globales de la mesa y pasan a dos salidas: una que elige las acciones y otra que estima el valor. En la versión RecurrentPPO se agrega una LSTM de 128 unidades antes de esas salidas.

4.5 Entrenamiento

- Algoritmo: PPO y RecurrentPPO. PPO es estable con varias acciones a la vez y no deja que la política cambie demasiado de golpe.
- Pasos: 5 millones. Las curvas se estabilizan entre 1.5 y 2 millones, el resto sirve para afinar.
- Mesas en paralelo: 8, para juntar más experiencia por segundo.
- Gamma: 0.995, porque detectar a un contador toma decenas de rondas.
- n_steps, batch y epochs: 2048, 2048 y 5. Con estos valores RecurrentPPO pasó de 10 horas a 4.5 horas de entrenamiento.
- ent_coef: 0.01. Con 0.001 el agente nunca expulsó a nadie en el primer millón de pasos.
- VecNormalize: normaliza observaciones y recompensas porque las variables tienen escalas muy distintas.
- Evaluación cada 100 mil pasos, guardando los modelos best, last y final.

4.6 Baselines

- Regla fija: expulsar si la correlación entre apuesta y conteo pasa de un umbral después de 30 rondas. Es lo que haría un supervisor con una regla simple.
- Clasificadores supervisados (regresión logística y gradient boosting) entrenados con las etiquetas reales del simulador, que el agente de RL no tiene. Expulsan si la probabilidad de ser contador pasa de un umbral.

4.7 Extensiones

- Curriculum de camuflaje: el nivel de camuflaje del disimulado va de 0 (contador abierto) a 1 (casi imposible de distinguir) y sube por etapas cuando el agente detecta suficientes disimulados.
- Adversario evolutivo: CMA-ES ajusta el camuflaje del disimulado para que gane lo más posible sin ser expulsado. En una primera prueba contra la regla fija, el disimulado pasó de ser expulsado el 52% de las veces al 16%. Aprendió a subir su apuesta más tarde y a cometer errores que le cuestan poco.
- Reentrenamiento continuo: en cada ciclo el adversario mejora y el supervisor se vuelve a entrenar con 70% de camuflajes nuevos y 30% anteriores para no olvidarlos. El modelo nuevo solo se acepta si su F1 sube al menos 0.01 y los falsos positivos se quedan en 5% o menos.
- Mesa visual con explicaciones: el script render_table.py graba un episodio y genera una repetición en HTML con las cartas, las apuestas, el conteo y cada expulsión marcada como acierto o error. En cada silla muestra las tres variables que más influyeron en la decisión.

5. Herramientas aplicadas

5.1 Herramientas vistas en clase

- Procesos de decisión de Markov y POMDP para plantear el problema.
- Métodos de gradiente de política y PPO.
- Diseño de la función de recompensa y cómo cambia el comportamiento del agente.
- Python, Gymnasium y PyTorch para el entorno y la red.

5.2 Herramientas no vistas en clase

Stable-Baselines3 y sb3-contrib. Son bibliotecas con algoritmos de RL ya probados, construidas sobre PyTorch. Se usó PPO de SB3 y RecurrentPPO de sb3-contrib con una política propia. SubprocVecEnv corre 8 mesas en procesos separados y VecNormalize normaliza observaciones y recompensas. Sus estadísticas se guardan junto al modelo para usarlas al evaluar.

RecurrentPPO. Es PPO con una LSTM dentro de la política. La LSTM guarda un resumen de lo que el agente ha visto, lo que le da memoria en un problema parcialmente observable.

CMA-ES. Es un algoritmo evolutivo que optimiza sin usar gradientes. En cada generación prueba varios candidatos, se queda con los mejores y ajusta su búsqueda hacia ellos. Se usó porque el

resultado del adversario solo se puede medir simulando partidas. Los mejores adversarios de cada ciclo se guardan en una liga para que el supervisor no olvide cómo detectarlos.

Gradientes integrados. Es un método para explicar qué tanto influye cada variable de entrada en la decisión de una red. Aquí se usó para explicar por qué el agente decide expulsar a alguien. Para el clasificador se usó importancia por permutación.

Gradient boosting y regresión logística (scikit-learn). Son los clasificadores usados como comparación. El gradient boosting combina muchos árboles de decisión pequeños, donde cada uno corrige los errores de los anteriores.

Curriculum learning. Consiste en entrenar con tareas cada vez más difíciles. Aquí se sube el nivel de camuflaje del disimulado cuando el agente ya domina el nivel actual.

Otras herramientas: TensorBoard para seguir el entrenamiento, pytest para las pruebas y Git con GitHub para trabajar en paralelo.

5.3 Uso de IA generativa

Se usó un asistente de IA generativa (Claude) para apoyar el diseño, la programación y la corrección de errores. Todas sus propuestas se revisaron con pruebas automáticas y simulaciones antes de aceptarlas. Estos fueron los usos más importantes:

- Diseño del proyecto. Se le pidió una arquitectura para el proyecto y una forma de dividir el trabajo. Propuso el entorno en Gymnasium, el formato SeatEvent y la división entre simulación y supervisor. Fue un buen punto de partida, aunque se simplificó la configuración.
- Contador disimulado. Se le pidió programar al disimulado. Su primera versión tenía un error: con mucho camuflaje el jugador perdía dinero. Lo detectamos con nuestras simulaciones y lo corregimos haciendo que solo cometa errores donde casi no le cuestan.
- Agente que no expulsaba. Cuando el agente PPO no expulsaba a nadie, sugirió bajar ent_coef. Esa sugerencia empeoró el problema, así que volvimos al valor original y el agente salió solo de ese comportamiento cerca del millón de pasos.
- Reward hacking. Le pedimos revisar si el agente estaba haciendo trampa. Encontró que la duración distinta de las sesiones le daba una pista falsa, y la corregimos dando la misma duración a todos.
- Velocidad de entrenamiento. Sugirió cambiar algunos parámetros de RecurrentPPO, lo que bajó el tiempo de 10 a 4.5 horas sin perder calidad.

Cómo cambió la idea. Al principio queríamos un supervisor con memoria (RecurrentPPO). Con los experimentos cambiamos tres cosas. Primero, las variables por silla fueron suficientes y PPO sin memoria funcionó mejor y cuatro veces más rápido. Segundo, hubo que corregir el simulador antes de confiar en los resultados. Tercero, agregamos clasificadores con etiquetas reales para medir si el RL aportaba algo. La IA ayudó a escribir código más rápido y dio buenas ideas, pero se equivocó cuando algo no se podía comprobar sin simular. Por eso cada cambio se validó con pruebas y las decisiones finales se tomaron con los datos medidos.

6. Resultados

Se midió el porcentaje de contadores y disimulados expulsados (recall), el porcentaje de casuales expulsados por error (falsos positivos, con un límite de 5%), el F1, las rondas que tarda en detectar y la recompensa por cada 100 rondas. Todos los supervisores se evaluaron con las mismas 20 semillas y con sesiones de 100 rondas. Los modelos de RL se entrenaron 5 millones de pasos con 8 mesas.

Supervisor	**Recall contadores**	**Recall disimulados**	**Falsos positivos**	**F1**	**Rondas a detección**	**Recompensa por 100 rondas**
Nunca expulsa	0%	0%	0%	0.00	-	-0.9
Regla fija con Pearson	65%	66%	2.7%	0.76	-	+15.6
Regla fija con Spearman	63%	61%	0.9%	0.75	-	+16.7
Regresión logística	65%	71%	1.1%	0.80	-	+18.0
Gradient boosting	71%	78%	0.7%	0.85	31	+20.5
PPO	80.5%	80.4%	2.7%	0.862	20	+22.9
RecurrentPPO con curriculum	79%	71.5%	1.7%	0.841	24	+16.8

- PPO fue el mejor en recall, F1, recompensa y velocidad. Detecta en 20 rondas, contra 31 del mejor clasificador, y lo hace sin etiquetas. A cambio tiene más falsos positivos que el clasificador (2.7% contra 0.7%), pero sigue dentro del límite de 5%. Detectar 11 rondas antes vale más que ese riesgo extra.
- La memoria no ayudó. Las variables ya resumen 50 rondas, así que la LSTM aporta poco, es cuatro veces más lenta y detecta peor a los disimulados.
- En la mesa visual, durante un turno de 600 rondas, el PPO expulsó a 7 contadores y a ningún casual.

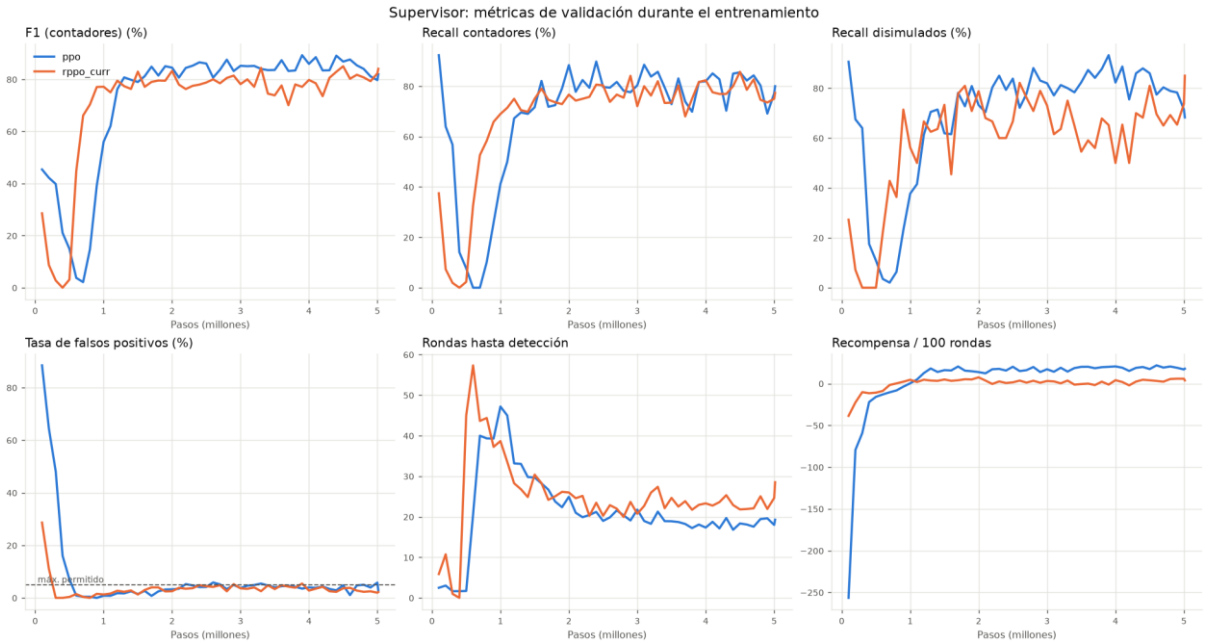


Figura 1. Métricas de validación durante el entrenamiento de PPO y RecurrentPPO con curriculum.

Cómo aprendió. La Figura 1 muestra el mismo patrón en los dos modelos. Al principio el agente expulsa de más (89% de falsos positivos en PPO). Luego deja de expulsar por completo, porque la penalización de -25 hace que no expulsar parezca lo más seguro. Cerca del millón de pasos aprende a expulsar con criterio. Desde los 2 millones de pasos los resultados se estabilizan, con falsos positivos cerca del 5%.

Qué mira el agente. Según los gradientes integrados, promediados sobre 50 expulsiones correctas, las variables que más llevan al agente a expulsar son:

- index_play_rate, con +7.6.
- basic_strategy_error_rate, con +4.9.
- corr_bet_tc, con +4.2.
- rank_corr_bet_tc, con +0.9.

El agente combina cómo juega la persona (jugadas índice y errores) con cómo apuesta (relación con el conteo). En cambio, el gradient boosting depende casi solo de los errores de estrategia básica.

6.1 Limitaciones

- Jugar perfecto delata. La tasa de errores de estrategia básica casi basta por sí sola para separar a los grupos. Falta probar el agente sin esa variable y dejar que el adversario aprenda a equivocarse.
- El curriculum no pasó de la primera etapa. Pedía detectar el 90% de los disimulados y el agente llegó a cerca del 82%.
- El reentrenamiento continuo está programado y probado, pero no se corrió completo porque toma varias horas.
- Se usó una sola semilla de entrenamiento por configuración. El modelo best se eligió con pocos episodios y es poco confiable, por eso se reporta el modelo final.

- Vigilar solo tiene un costo y no le da información extra al agente.

7. Conclusión

Un agente de aprendizaje por refuerzo puede hacer el trabajo de un supervisor de mesa mejor que una regla fija y mejor que un clasificador entrenado con etiquetas que en la vida real no existen. El agente PPO detectó al 80% de los contadores, abiertos y disimulados, con solo 2.7% de falsos positivos y 11 rondas antes que el mejor clasificador. Eso le dio la mayor recompensa económica. Su ventaja viene de tratar el problema como lo que es: una serie de decisiones con costos, donde hay que saber cuándo esperar y cuándo actuar.

El proyecto también mostró que en RL el diseño del entorno y de la recompensa importa tanto como el algoritmo. Una diferencia en la duración de las sesiones bastó para que el agente encontrara un atajo, y el castigo por expulsar a un casual define cuántos errores acepta. Las variables hechas a mano hicieron innecesaria la memoria, y la explicabilidad confirmó que el agente decide con señales que tienen sentido. Los siguientes pasos son reentrenar con el adversario evolutivo, ajustar las metas del curriculum y probar el agente sin la variable de errores de estrategia.

8. Repositorio

https://github.com/Gerax5/PROYECTO-RL

Bibliografía

Bengio, Y., Louradour, J., Collobert, R. y Weston, J. (2009). Curriculum learning. En *Proceedings of the 26th International Conference on Machine Learning* (pp. 41-48).

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189-1232.

Hansen, N. (2016). The CMA evolution strategy: A tutorial. arXiv:1604.00772.